AMAZON BEDROCK ／ デスクトップアプリの CLAUDE CODE

複雑な有価証券報告書データを、AI で数分でダッシュボードに

Claude Code ハンズオン研修

題材：**EDINET の XBRL**

ゴール：**財務ダッシュボード**

体験：**スキル化 → 横展開**

AGENDA

本日の流れ

1. 配布物と作業フォルダの設定
2. まず触ってみる
3. 今回のゴール
4. テーマの説明（ XBRL ）
5. 実際にやってみる
6. スキル
7. 人が気づけない差にも気づく（会計基準）
8. CLAUDE.md とメモリ
9. 自分でやってみる（課題）
10. まとめ

INTRODUCTION

講師 自己紹介

お名前 ／ 所属・役割 ／ Claude ・生成 AI でやってきたこと



今回の目的

今日のゴールは、「難しい・面倒」を AI に任せ、

自分の手元で成果物まで作りきる感覚を持ち帰る。



難しい処理も AI に任せる

調べる・場合分け・整形まで、まとめてお願いできる



自分の PC ・自分の業務で

特別な基盤はいらない。手元のファイルとコマンドで完結



作った手順は資産になる

スキル化して別の会社・別の業務へ横展開

実行環境 ── モデルは AWS の Bedrock で動く

Amazon Bedrock

（AWS のフルマネージド生成 AI 基盤）

東京リージョン ／ 認証は Entra ID 連携

🧠 考える ＝ クラウド (Bedrock)

モデルはすべて AWS 上で実行される

🛠️ 動かす ＝ 手元 PC

ファイル操作・コマンドは自分の PC で実行

制約： MCP を利用した外部サービス連携は不可。

OVERVIEW

全体の流れ

ハンズオン 全体の流れ

準備 → まず触る → ゴール共有 → XBRLを知る → 作る（1社目） → スキル化して横展開

1 準備

配布物・データを共有
作業フォルダを Claude Code に教える

2 まず触る

軽いお題で一度動かす
まず Claude に聞く／対話で育てる

3 ゴール

有価証券報告書 → 財務ダッシュボード
完成イメージを先に見せる

4 XBRLを知る

覚えなくてよい
データの中身を実物で確認

5 作る（1社目）

XBRL → data.json → dashboard.html
試行錯誤で仕上げる

6 スキル化→横展開

手順をスキルに固める
会計基準が違っても**自動で対応**

体感のヤマ場
再利用でこれだけ短縮

1社目：試行錯誤で完成
プロンプトを直しながら

2社目：数分で再現
スキルを適用（別の会計基準もOK）

今後：CLAUDE.md・メモリで定着／API で多数の企業へ

配布物の共有と、作業フォルダの設定

配布物

演習で使う

▶ Code用サンプルプロンプト集

参考資料（読む用）

- XBRL 入門 .html
- スキルとスラッシュコマンド .html
- モデル選択の考え方 .html
- 用語集 .html
- Code 用サンプルプロンプト .html

作業フォルダの設定：配布データのフォルダを Claude Code に教える（以降はそのフォルダ基点で指示できる）

作業フォルダを Claude Code に教える



デスクトップアプリで、配布フォルダ（ハンズオン）を作業フォルダに指定する（赤枠）

まず触ってみる

▶ サンプルプロンプト集

プロンプト

```
金の重量 (g) と純度 (%) を入力すると買取概算額を計算できる、単一 HTML のツールを作ってください。
・外部ライブラリは使わず、 1 ファイル (index.html) で完結
・入力はフォーム、計算はその場で表示
・グラム単価は仮に  9,000 円 /g  をコード内の定数に
作成後、ローカルで開いて動作を確認する手順も教えてください。
```

参考： 困った時の助け舟

— まず Claude に聞いてみる

— 完璧なプロンプトを目指さない（対話で育てる）

— 確認が入ることがある ← AI の時代でも人間が判断する場面

まず触ってみる


金 買取概算計算ツール
ファイル /Users/z2200318/P...
金 買取概算計算ツール
重量と純度を入力すると、買取のおおよその金額を計算します。
重量 (g)
例: 10.5
グラム単位で入力してください
純度 (%)
例: 75 (K18相当)
K24=100%, K18≈75%, K14≈58.5%
概算額を計算する
適用単価: ¥9,000/g

今回のゴール

EDINET にある複雑な有価証券報告書ファイルから、**綺麗な財務ダッシュボード**を作る。

—完成イメージ →（右のサンプル）

—人が扱うには複雑すぎる XBRL を AI でまとめて可視化

商用イメージの参照：[バフェットコード]

Copilot との住み分け： 市民開発・ Office 連携は Copilot ／自律エージェント・複雑データ処理・コード生成は Claude Code 。



完成ダッシュボード（ＡＲＥホールディングス）

そもそも XBRL とは?

有価証券報告書などの数字を、**コンピュータが読めるようにタグ付けした形式**。

👤 人にとっては

—タグや定義が大量で読みづらい

—「どの数字が何か」が一目で分からない

🤖 AI にとっては

—タグが付いているので機械処理しやすい

—狙った数字を正確に取り出せる

＝人には辛い形式だからこそ、AI が力を発揮する。

XBRL の構成



—タクソノミ（辞書・型）＋インスタンス（実データ .xbrl ）＝ XBRL

—数値の実体はファクト。「どの期・連結 / 個別か」はコンテキストで決まる

—取得は 手段 1 ＝ Web 手動 DL （キー不要）

1 つのデータはこう書かれている

```
<jpcrp_cor:RevenueIFRSSummaryOfBusinessResults
    contextRef="CurrentYearDuration" unitRef="JPY">569992000000</…>
```

要するに → **「当期の売上は 約 5,700 億円」** と書いてあるだけ。

— 「いつの数字か」（当期／前期）と「単位」（円）が、数字と別の場所にタグで付いている

— 円で書かれているので、ダッシュボードでは ÷1 億して「億円」で表示する

— 1 つ 1 つは単純。でも種類が多く別々の場所にあるので、人が全部たどるのは大変 → だから AI に任せる

05 ・ 演習

実際にやってみる

ここからは配布データを使って、自分の手で動かします

プロンプトで複雑な処理を Claude に実行させる

▶ サンプルプロンプト集

プロンプト

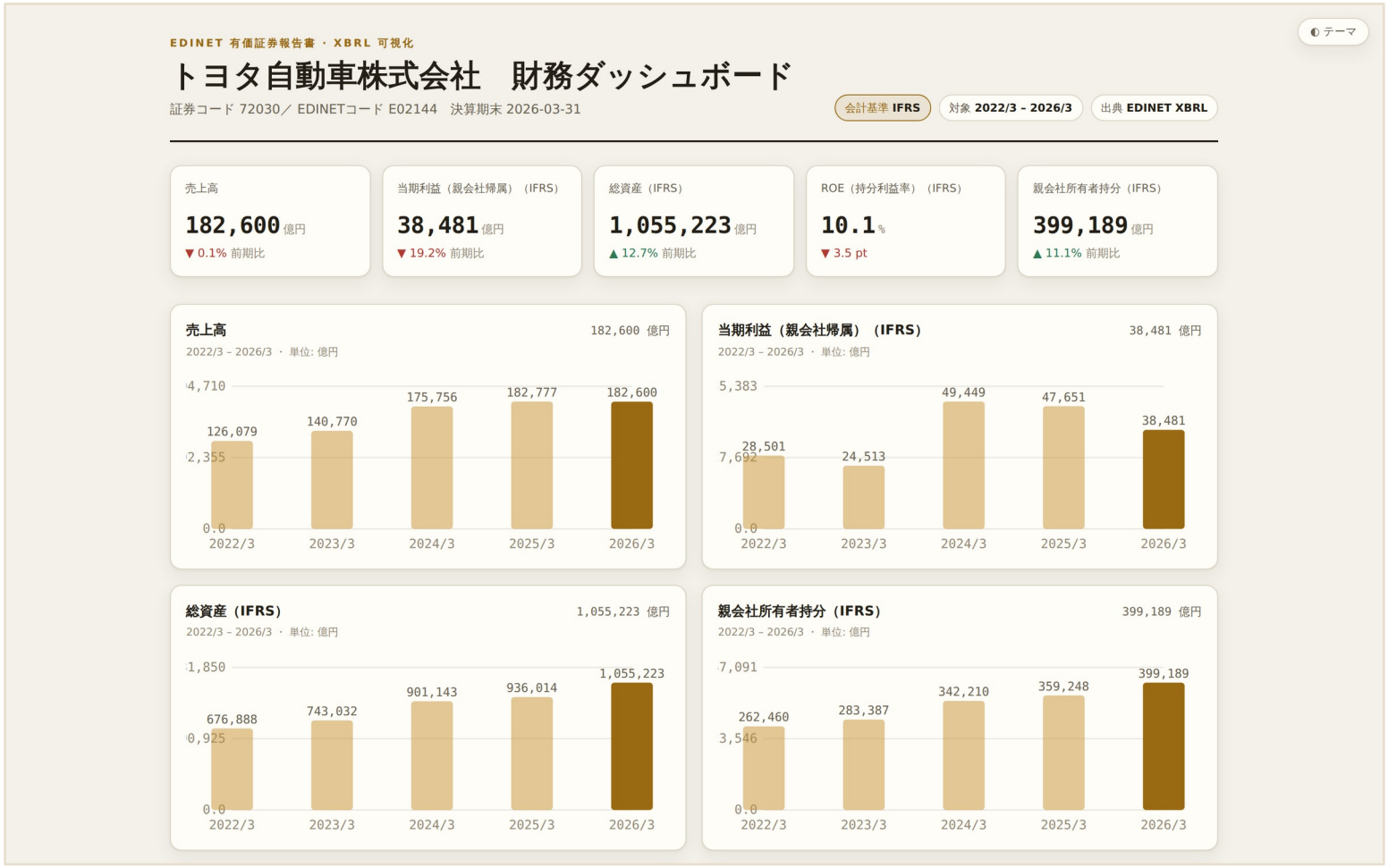
data/ トヨタ自動車 _E02144 の有価証券報告書 XBRL から主要財務（連結・通期）を抽出して data.json にまとめ、その data.json から外部ライブラリなしの単一 HTML ダッシュボード（dashboard.html）を作って。抽出・生成はスクリプトで行い、結果を見ながら仕上げて。

この指示で、裏では次の処理が自動で走る：



コスト作法： 巨大な XBRL 本文を会話に貼らない。構成→対象タグ→スクリプト化→結果だけ確認。

できあがるのは、こんなダッシュボード



トヨタ自動車株式会社（IFRS ・連結）　｜　サンプル：配布 / サンプル / トヨタ自動車 _ 財務ダッシュボード .html

別の会社も、すぐ作れる

「次はソニーで」「次は日立で」— **何社もやりたくなりますよね。**

一度やった作業は…

— 1 社目は試行錯誤

— でも手順はもう分かっている

— 毎回同じ指示を打ち直すのは大変

→ 手順を「道具」にまとめて再利用

スキル

という仕組み。 2 社目以降は数分。

スキル＝手順書＋道具のパッケージ

— 作業の「やり方 (手順書) 」と「道具 (スクリプト) 」を
1 セットに

— 必要なときに Claude が自動で読み込んで実行

— 一度作れば、別の会社でもそのまま再利用・横展開

— 属人化を防ぐ“内製の資産”になる

```
skills/edinet-xbrl-dashboard/
├─ SKILL.md      … 使い方の説明書（本体）
├─ scripts/       … 道具（Python）
│   ├─ extract_financials.py
│   └─ build_dashboard.py
└─ assets/dashboard_template.html
```

スキルは、プロンプトで作れる

▶ サンプルプロンプト集

プロンプト

**今やった「有価証券報告書 XBRL → 主要財務の抽出 → ダッシュボード生成」の一連の手順を、
再利用できるスキルにまとめてください。
・入力（XBRL フォルダ）と出力（dashboard.html）を明記
・会社固有の値はハードコードせず、別の会社でもそのまま使えるように
作成後、このスキルの使い方（呼び出し方）も教えてください。**

ポイント

―試行錯誤の成果を、そのまま“再利用できる資産”に変える

―次からは「このスキルで、この会社を」と一言でよい

スキルで、別の会社も（実行結果）

▶ 実行結果のスクリーンショットをここに

（スキルで別の会社を出した結果・後日差し替え）

ちなみに… 会計基準は 1 種類じゃない

別の会社でやってみると――

同じ「売上」でも、会社によって科目名が違う。

IFRS 採用の会社

―売上収益 / 税引前利益 / 親会社所有者持分
…

日本基準 採用の会社

―売上高 / 経常利益 / 純資産 …

配布 17 社にも、IFRS と日本基準が混在している。

会計基準が違っても、 Claude がそのまま対応

人がやると…

—基準を判定し、拾う項目を切り替える

—銀行など特殊な様式にも対応…

—知らずに実行して、エラーで気づく

Claude なら…

—どの会社を指しても同じ操作で OK

—違いに“気づいて”対応してくれる

—IFRS の会社も銀行も、続けて流せば両方出る

立て付け

—人間は「知らずに実行 → エラー」になりがちなところを、 Claude は先に気づいて確認してくれる

—締めに「１社だけ原文と検算」＝任せつつ、人が確かめる

気づいた対応は、スキルに反映する

▶ サンプルプロンプト集

プロンプト

**日本基準の会社で必要になった科目の対応を、 edinet-xbrl-dashboard スキルに反映して、
次からは自動で切り替わるようにしてください。**

スキルが“育つ”

—対応を一度スキルに書けば恒久化

—次の会社からは、もう気にしなくてよい

これが内製の効き目

—使うほどスキルが賢くなる

—チームの“資産”として貯まっていく

よりよいアウトプットのための仕組み

毎回うまく動かすために、 Claude に “覚えさせる” 仕組みが 2 つある。

CLAUDE.md

人が書く「決まり事」

—データの場所・採用ルール・出力先などを明記

—毎回指示しなくても品質が安定

メモリ

Claude が対話から自動で覚える

—与えた修正や好みを学習ノートに蓄積

—使うほど自分の好みに馴染む

CLAUDE.md

作業フォルダに置くと起動時に自動で読み込まれる「前提メモ」。

```
# このプロジェクトについて（例）
- 対象データ： EDINET の有報 XBRL （./data/…）
- 採用方針： 連結・通期を優先。表示は億円
- 出力： ./output/ に成果物（単一 HTML ）
- 会計基準は自動判定。銀行は解釈に注意
```

読み込み順（後ほど優先）

—管理ポリシー（組織）

—~/.claude/CLAUDE.md （個人）

—上位ディレクトリ

—./CLAUDE.md （プロジェクト）

—./CLAUDE.local.md

見せ方：「置く前／置いた後」で出力の質を比較。

メモリ（対話から自動で覚える）

```
~/.claude/projects/<repo>/memory/
├─ MEMORY.md          （インデックス）
└─ feedback_*.md  等  （トピック別）
```

—/memory で確認・編集。既定で ON 、マシンローカル

—例：「金額は億円」と一度直すと次から反映される

CLAUDE.md との使い分け

	CLAUDE.md	メモリ
書き手	人	Claude
中身	前提・ルール	学習・好み
性質	git で共有・再現	ローカル・自動

再現性は **CLAUDE.md ＋スキル**、快適さは **メモリ**。

課題 — 手を動かして広げる



好きな企業を追加

配布データ以外の会社でも
同じスキルで作ってみる



機能を足してみる

バフェットコードを参考に
指標やグラフを追加



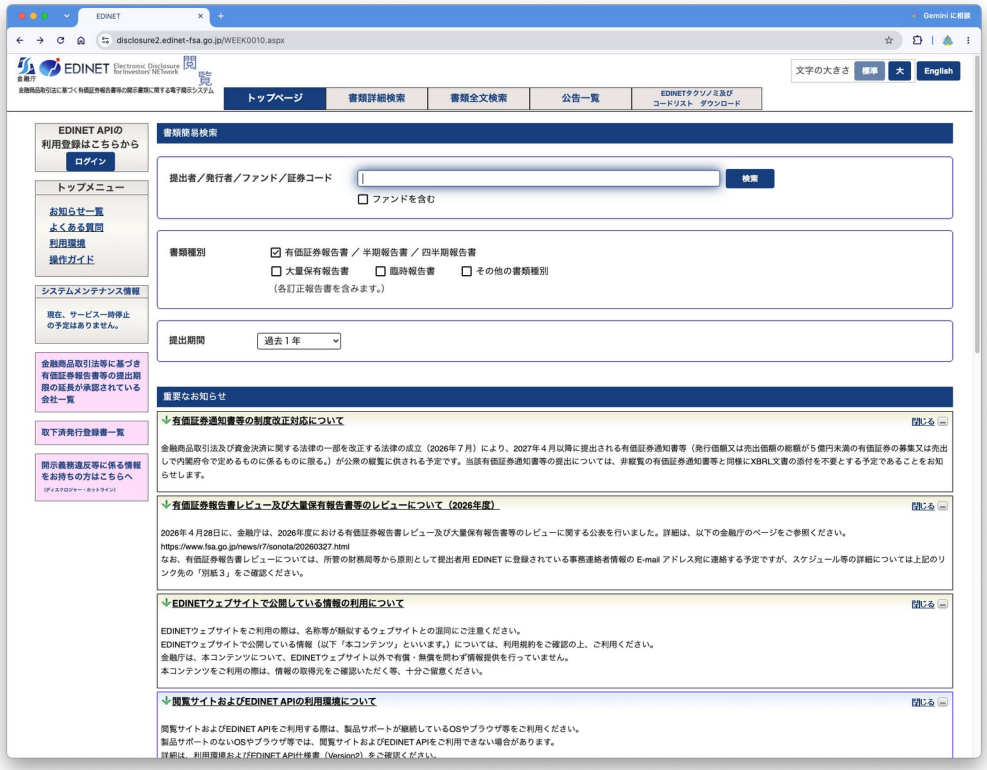
企業どうしを比較

2 社を並べて
強み・弱みを見比べる

配布外の会社は EDINET から取得

① 会社名・証券コードで検索（書類種別＝有価証券報告書）　② 一覧の「 XBRL 」から ZIP をダウンロード　③ 解凍して data/ に配置

🔗 https://disclosure2.edinet-fsa.go.jp/



① 書類簡易検索（トップページ）



② 「 XBRL 」からダウンロード

まとめ

— 人間が実行するには複雑な処理も、 Claude なら自動化できる

— アイデア次第で、普段の事務処理も大きく時間短縮できる

— ビジュアライズして、これまで気づけなかった価値を生み出せる

まず触る → 1 社作る → スキルで横展開 → 自分の業務へ